

Distillation of GPT-2 Small

1. Imports and Configuration

```
In [ ]: import copy
import os
import math
import time
import random

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from datasets import load_dataset
from tqdm.auto import tqdm
from transformers import GPT2Config, GPT2LMHeadModel, GPT2TokenizerFast
import matplotlib.pyplot as plt

def set_seed(seed):
    """Make results reproducible."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

CFG = {
    # Data
    "dataset_name": "Salesforce/wikitext",
    "dataset_config": "wikitext-2-raw-v1", # ~2.4M tokens; the only dataset used in this project
    "block_size": 1024, # tokens per text block
    "batch_size": 16, # blocks per training step (~32 GB peak VRAM at block_size 1024)

    # Teacher and student model size
    "teacher_name": "gpt2",
    "student_n_layer": 6,
    "student_n_embd": 384,
    "student_n_head": 6,

    # Training
    "seed": 42,
    "lr": 5e-4,
    "epochs": 50,
    "sweep_epochs": 5,
    "early_stopping_patience": 2, # stop if validation perplexity does not improve for this many epochs

    # Distillation
    "temperature": 1.0,
    "temperature_values": [1.0, 2.0, 4.0, 7.0, 10.0, 15.0],
    "alpha": 0.5,
}

set_seed(CFG["seed"])

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
if device == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))
```

Using device: cuda

GPU: AMD Radeon PRO W7900

2. Load the Tokenizer

```
In [2]: # Load the GPT-2 tokenizer
# tokenized returns token IDs, attention masks, etc. as a dictionary of lists.
tokenizer = GPT2TokenizerFast.from_pretrained(CFG["teacher_name"])

print("Vocabulary size:", tokenizer.vocab_size)
# Quick sanity check: encode then decode a sentence and confirm we get it back.
demo = tokenizer("This project is a demonstration of knowledge distillation for GPT-2")
print("Token IDs:", demo["input_ids"])
print("Decoded   :", tokenizer.decode(demo["input_ids"]))
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Vocabulary size: 50257

Token IDs: [1212, 1628, 318, 257, 13646, 286, 3725, 1233, 40903, 329, 402, 11571, 12, 17]

Decoded : This project is a demonstration of knowledge distillation for GPT-2

3. Load & Prepare the Data

We use `wikitext-2-raw-v1` for everything in this project: the temperature sweep, the final training of all three students, and all evaluation. It is about 2.4M GPT-2 tokens, which is small enough to train 11 models within a reasonable time budget.

The preparation has three steps:

1. **Tokenize** every line of text into token IDs.
2. **Group into fixed-length blocks** of `block_size` tokens. Language models train on fixed-length sequences, so we concatenate all the tokens into one long stream and then chop it into equal chunks. Any leftover tokens that don't fill a full block are dropped.
3. **Create `DataLoader`** for training, validation and testing

```
In [ ]: # 1) Download the raw dataset. It has "train", "validation", and "test" sets.
raw_datasets = load_dataset(CFG["dataset_name"], CFG["dataset_config"])
print(raw_datasets)

# 2) Tokenize text in batches.
def tokenize_batch(batch):
    return tokenizer(batch["text"])

tokenized_datasets = raw_datasets.map(
    tokenize_batch,
    batched=True,
    remove_columns=["text"],
)
print(tokenized_datasets)

# 3) Join tokenized text together and split it into fixed-size blocks.
block_size = CFG["block_size"]

def group_texts(batch):
    all_input_ids = []

    for token_list in batch["input_ids"]:
        all_input_ids.extend(token_list)

    total_length = (len(all_input_ids) // block_size) * block_size
    blocks = []

    for i in range(0, total_length, block_size):
        blocks.append(all_input_ids[i:i + block_size])

    return {"input_ids": blocks}

lm_datasets = tokenized_datasets.map(
    group_texts,
    batched=True,
    remove_columns=tokenized_datasets["train"].column_names,
)
lm_datasets.set_format(type="torch", columns=["input_ids"])

print(lm_datasets)

# 4) Create DataLoaders for each split. Each batch is a tensor with shape (batch_size, block_size).
train_loader = DataLoader(lm_datasets["train"], batch_size=CFG["batch_size"], shuffle=True)
val_loader = DataLoader(lm_datasets["validation"], batch_size=CFG["batch_size"])
test_loader = DataLoader(lm_datasets["test"], batch_size=CFG["batch_size"])
```

```

DatasetDict({
  test: Dataset({
    features: ['text'],
    num_rows: 4358
  })
  train: Dataset({
    features: ['text'],
    num_rows: 36718
  })
  validation: Dataset({
    features: ['text'],
    num_rows: 3760
  })
})
DatasetDict({
  test: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 4358
  })
  train: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 36718
  })
  validation: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 3760
  })
})
DatasetDict({
  test: Dataset({
    features: ['input_ids'],
    num_rows: 274
  })
  train: Dataset({
    features: ['input_ids'],
    num_rows: 2318
  })
  validation: Dataset({
    features: ['input_ids'],
    num_rows: 240
  })
})
})

```

4. Load the Teacher (GPT-2 Small)

The teacher is a pretrained GPT-2 small

```

In [4]: def count_parameters(model):
        total = 0
        for param in model.parameters():
            total += param.numel()
        return total

# Download/Load GPT-2 small.
teacher = GPT2LMHeadModel.from_pretrained(CFG["teacher_name"]).to(device)
teacher.eval() # freeze behaviour: no dropout, no parameter updates

teacher_params = count_parameters(teacher)
print(f"Teacher: {CFG['teacher_name']}")
print(f"Teacher parameters: {teacher_params}")
print(f"Teacher layers: {teacher.config.n_layer}, hidden size: {teacher.config.n_embd}")

```

```

Loading weights: 0% | 0/148 [00:00<?, ?it/s]
Teacher: gpt2
Teacher parameters: 124439808
Teacher layers: 12, hidden size: 768

```

5. Build Three Student Models (Same Architecture)

We create three students with the **same smaller GPT-2 architecture** so the comparison is fair. Only the training objective changes:

- **Teacher + corpus:** learns from both hard labels and teacher soft labels.
- **Corpus only:** learns only from the true next tokens in WikiText.
- **Teacher only:** learns only by matching the teacher's output distribution.

All three students use the same tokenizer and vocabulary as the teacher, which is required for comparing next-token distributions.

```

In [5]: def build_student():
        student_config = GPT2Config(
            vocab_size=teacher.config.vocab_size, # MUST match the teacher, number of tokens in the vocabulary
            n_positions=teacher.config.n_positions, # max context length (1024)

```

```

        n_ctx=teacher.config.n_positions,
        n_embd=CFG["student_n_embd"],          # hidden size (width)
        n_layer=CFG["student_n_layer"],        # number of transformer blocks (depth)
        n_head=CFG["student_n_head"],          # attention heads
        bos_token_id=tokenizer.bos_token_id,    # token ID for the beginning-of-sequence token
        eos_token_id=tokenizer.eos_token_id,    # token ID for the end-of-sequence token
    )
    return GPT2LMHeadModel(student_config).to(device)

def build_seeded_student():
    # Build each student from the same initial weights for a fair comparison.
    set_seed(CFG["seed"])
    return build_student()

student_models = {
    "Teacher + corpus": build_seeded_student(),
    "Corpus only": build_seeded_student(),
    "Teacher only": build_seeded_student(),
}

student_params = count_parameters(student_models["Teacher + corpus"]) # all students have the same number of parameters, so we can just count one
print(f"Student parameters: {student_params:,}")
print(f"Student layers: {CFG['student_n_layer']}, hidden size: {CFG['student_n_embd']}")
print(f"Compression ratio (teacher / student): {teacher_params / student_params:.1f}x")

```

Student parameters: 30,339,456

Student layers: 6, hidden size: 384

Compression ratio (teacher / student): 4.1x

6. Loss Functions For The Three Training Methods

This project compares three ways to train the same student architecture:

- **Teacher + corpus:** uses both the real corpus labels and the teacher's soft targets.
- **Corpus only:** uses only the real next-token labels from the dataset.
- **Teacher only:** uses only the teacher's soft targets.

In code, the combined method is:

```
total_loss = alpha * corpus_loss + (1 - alpha) * teacher_loss
```

The loss design follows ideas from Hinton, Vinyals, and Dean's paper **Distilling the Knowledge in a Neural Network**:

Hard and soft losses

- `corpus_loss` is the hard-label loss. It uses cross-entropy against the true next token from the dataset.
- `teacher_loss` is the soft-label loss. It uses KL divergence to match the teacher's probability distribution.
- The paper explains that combining hard labels and soft teacher targets can improve the student.

Temperature

- Temperature softens the teacher's probability distribution.
- A higher temperature makes the probabilities less sharp and reveals more information about the teacher's uncertainty.
- In this project, we run a temperature sweep before final training to choose a good value.

T squared scaling

- The paper notes that gradients from soft targets shrink when temperature increases.
- To compensate, the distillation loss is multiplied by `temperature ** 2`.
- In the code, this scaling is applied inside `teacher_loss`.

```

In [6]: def compute_training_loss(student_logits, teacher_logits, input_ids, method):
        # This function computes the loss for one training batch.

        # student_logits: (batch_size, block_size, vocab_size) tensor of scores for every possible next token
        # we extract the vocab_size
        vocab_size = student_logits.size(-1)

        # GPT predicts the next token, so we compare position 0 with Label 1, position 1 with Label 2, etc.
        # take all batches, take all positions except the last one, take all vocabulary scores
        student_next_token_scores = student_logits[:, :-1, :].contiguous()
        # take all batches, take tokens from position 1 onward
        true_next_tokens = input_ids[:, 1:].contiguous()

        # Corpus Loss: student vs. the real next token from the dataset.
        # corpus_loss = average of -log(student probability assigned to each true next token)
        # cross entropy expect raw logits
        corpus_loss = F.cross_entropy(
            student_next_token_scores.view(-1, vocab_size), # reshape [batch_size * (block_size - 1), vocab_size]

```

```

        true_next_tokens.view(-1),                # reshape [batch_size * (block_size - 1)]
    )

    # Teacher Loss: student vs. the teacher's full probability distribution.
    teacher_loss = torch.zeros(), device=student_logits.device
    if teacher_logits is not None: ## no teacher_logits for corpus only training
        teacher_next_token_scores = teacher_logits[:, :-1, :].contiguous()

        student_log_probs = F.log_softmax(student_next_token_scores / CFG["temperature"], dim=-1)
        teacher_probs = F.softmax(teacher_next_token_scores / CFG["temperature"], dim=-1)

        teacher_loss = F.kl_div(
            student_log_probs.view(-1, vocab_size),
            teacher_probs.view(-1, vocab_size),
            reduction="batchmean",
        ) * (CFG["temperature"] ** 2)

    # Choose which training method this student uses.
    if method == "combined":
        total_loss = CFG["alpha"] * corpus_loss + (1 - CFG["alpha"]) * teacher_loss
    elif method == "corpus_only":
        total_loss = corpus_loss
    elif method == "teacher_only":
        total_loss = teacher_loss
    else:
        raise ValueError(f"Unknown training method: {method}")

    return total_loss, corpus_loss, teacher_loss

```

7. Training

7.1 Perplexity Evaluation

Perplexity measures how well the model predicts the next token. It is calculated from the average cross-entropy loss:

`perplexity = exp(average cross-entropy loss)`

```

In [7]: def evaluate_perplexity(model, loader):
        model.eval()
        total_loss = 0.0
        total_tokens = 0

        with torch.no_grad():
            for batch in loader:
                input_ids = batch["input_ids"].to(device)

                # Hugging Face computes the cross-entropy internally when labels are provided.
                # outputs.loss is the average negative log probability of the correct next tokens.
                outputs = model(input_ids=input_ids, labels=input_ids)

                num_tokens = input_ids.size(0) * (input_ids.size(1) - 1)
                total_loss = total_loss + outputs.loss.item() * num_tokens
                total_tokens = total_tokens + num_tokens

        average_loss = total_loss / total_tokens # average log perplexity
        return math.exp(average_loss) # convert back to perplexity

```

7.2 Model Training

Main model training loop with early stopping.

After every epoch the student is scored on the validation set. If validation perplexity has not improved for `early_stopping_patience` epochs, training stops.

Early stopping by itself would leave the model sitting on the epoch that *failed* to improve, which is one or two epochs past its best. We therefore keep a copy of the weights from the best epoch and restore them before returning, so the model that gets saved, evaluated on the test set, and used for text generation is the same model that early stopping selected. This also makes the final students consistent with the temperature sweep, which ranks temperatures by their best epoch.

```

In [ ]: def train_model(model, name, method):
        set_seed(CFG["seed"])

        optimizer = torch.optim.AdamW(model.parameters(), lr=CFG["lr"])
        loss_history = []
        validation_history = []

        best_validation_ppl = None
        best_epoch = None
        best_state = None
        epochs_without_improvement = 0

        for epoch in range(CFG["epochs"]):

```

```

model.train()
progress = tqdm(train_loader, desc=f"{name}: epoch {epoch + 1}/{CFG['epochs']}")

total_epoch_loss = 0
num_batches = 0

for batch in progress:
    input_ids = batch["input_ids"].to(device)

    # Run the student, get the final output scores (Logits) for every possible next token.
    student_logits = model(input_ids=input_ids).logits

    # Run the teacher only when this method needs teacher guidance.
    teacher_logits = None
    if method in ["combined", "teacher_only"]:
        with torch.no_grad():
            teacher_logits = teacher(input_ids=input_ids).logits

    loss, corpus_loss, teacher_loss = compute_training_loss(
        student_logits,
        teacher_logits,
        input_ids,
        method,
    )

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_epoch_loss = total_epoch_loss + loss.item()
    num_batches = num_batches + 1
    progress.set_postfix(loss=f"{loss.item():.3f}")

average_epoch_loss = total_epoch_loss / num_batches
loss_history.append(average_epoch_loss)
print(f"{name} average training loss after epoch {epoch + 1}: {average_epoch_loss:.3f}")

validation_ppl = evaluate_perplexity(model, val_loader)
validation_history.append(validation_ppl)
print(f"{name} validation perplexity after epoch {epoch + 1}: {validation_ppl:.2f}")

if best_validation_ppl is None or validation_ppl < best_validation_ppl:
    best_validation_ppl = validation_ppl
    best_epoch = epoch + 1
    # Snapshot the weights, not just the score. Training continues from here, so without a
    # copy these values would be overwritten by the next optimizer step.
    best_state = copy.deepcopy(model.state_dict())
    epochs_without_improvement = 0
else:
    epochs_without_improvement = epochs_without_improvement + 1

if epochs_without_improvement >= CFG["early_stopping_patience"]:
    print(f"Early stopping {name}: validation perplexity did not improve.")
    break

# Early stopping leaves the model on the epoch that failed to improve, which is 1-2 epochs past
# its best. Roll back so the model we save and evaluate is the one early stopping selected.
if best_state is not None:
    model.load_state_dict(best_state)
    print(f"Restored {name} to epoch {best_epoch} (validation perplexity {best_validation_ppl:.2f}")

print(f"Finished training {name} model.")
return loss_history, validation_history

```

7.3 Temperature Sweep Before Final Training

- Before final training, we try several temperature values: 1, 2, 4, 7, 10, 15.
- Each sweep run trains a throwaway student with `method="teacher_only"`, so the teacher's soft targets are the only training signal.
- The sweep uses validation perplexity to choose a good temperature.
- After the sweep, `CFG["temperature"]` is set to the best temperature.
- Fewer epochs are used to save run time, so we expect these perplexities to be higher than the final models.

```

In [9]: temperature_values = CFG["temperature_values"]
original_epochs = CFG["epochs"]
sweep_epochs = CFG["sweep_epochs"]

sweep_rows = []

for temperature in temperature_values:
    print(f"\n=== Temperature sweep: T = {temperature} ===")

    CFG["temperature"] = temperature

```

```

CFG["epochs"] = sweep_epochs

sweep_student = build_seeded_student()
# Sweep with the teacher signal alone so the corpus loss does not dilute the temperature effect.
sweep_loss_history, sweep_validation_history = train_model(
    sweep_student,
    name=f"Temperature {temperature}",
    method="teacher_only",
)

best_validation_ppl = min(sweep_validation_history)
final_validation_ppl = sweep_validation_history[-1]

sweep_rows.append(
    {
        "Temperature": temperature,
        # This loss is KL * T^2, so it grows with T by construction and must not be compared
        # across rows. Use the validation perplexity columns to rank temperatures.
        "Final training loss (KL x T^2)": sweep_loss_history[-1],
        "Best validation perplexity": best_validation_ppl,
        "Final validation perplexity": final_validation_ppl,
    }
)

del sweep_student
if device == "cuda":
    torch.cuda.empty_cache()

# Pick the temperature with the lowest best validation perplexity.
temperature_sweep = pd.DataFrame(sweep_rows)
best_temperature = temperature_sweep.loc[temperature_sweep["Best validation perplexity"].idxmin(), "Temperature"]

# Restore the normal epoch count, but keep the best temperature for final training.
CFG["epochs"] = original_epochs
CFG["temperature"] = float(best_temperature)

print(f"\nBest temperature from sweep: {CFG['temperature']}")
temperature_sweep

```

=== Temperature sweep: T = 1.0 ===

Temperature 1.0: epoch 1/5: 0% | 0/145 [00:00<?, ?it/s]

c:\Users\Ethan\AppData\Local\Programs\Python\Python312\Lib\site-packages\transformers\integrations\sdpa_attention.py:92: UserWarning: Using AOTrit on backend for Efficient Attention forward... (Triggered internally at C:/b/pytorch/aten/src/ATen/native/transformers/hip/attention.hip:1452.)

attn_output = torch.nn.functional.scaled_dot_product_attention(
c:\Users\Ethan\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\autograd\graph.py:841: UserWarning: Using AOTriton backend for Efficient Attention backward... (Triggered internally at C:/b/pytorch/aten/src/ATen/native/transformers/hip/attention_backward.hip:551.)

return Variable._execution_engine.run_backward(# Calls into the C++ engine to run the backward pass

[transformers] `loss\_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.

Temperature 1.0 average training loss after epoch 1: 3.891

Temperature 1.0 validation perplexity after epoch 1: 735.70

Temperature 1.0: epoch 2/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 1.0 average training loss after epoch 2: 2.986

Temperature 1.0 validation perplexity after epoch 2: 478.31

Temperature 1.0: epoch 3/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 1.0 average training loss after epoch 3: 2.636

Temperature 1.0 validation perplexity after epoch 3: 382.64

Temperature 1.0: epoch 4/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 1.0 average training loss after epoch 4: 2.400

Temperature 1.0 validation perplexity after epoch 4: 311.84

Temperature 1.0: epoch 5/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 1.0 average training loss after epoch 5: 2.185

Temperature 1.0 validation perplexity after epoch 5: 258.12

Restored Temperature 1.0 to epoch 5 (validation perplexity 258.12)

Finished training Temperature 1.0 model.

=== Temperature sweep: T = 2.0 ===

Temperature 2.0: epoch 1/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 2.0 average training loss after epoch 1: 4.703

Temperature 2.0 validation perplexity after epoch 1: 1989.41

Temperature 2.0: epoch 2/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 2.0 average training loss after epoch 2: 3.715

Temperature 2.0 validation perplexity after epoch 2: 1017.87

Temperature 2.0: epoch 3/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 2.0 average training loss after epoch 3: 3.218

Temperature 2.0 validation perplexity after epoch 3: 769.50

Temperature 2.0: epoch 4/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 2.0 average training loss after epoch 4: 2.944

Temperature 2.0 validation perplexity after epoch 4: 611.15

Temperature 2.0: epoch 5/5: 0% | 0/145 [00:00<?, ?it/s]

Temperature 2.0 average training loss after epoch 5: 2.750

Temperature 2.0 validation perplexity after epoch 5: 549.96

Restored Temperature 2.0 to epoch 5 (validation perplexity 549.96)

Finished training Temperature 2.0 model.

=== Temperature sweep: T = 4.0 ===

Temperature 4.0: epoch 1/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 4.0 average training loss after epoch 1: 3.453
Temperature 4.0 validation perplexity after epoch 1: 3283.42
Temperature 4.0: epoch 2/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 4.0 average training loss after epoch 2: 3.147
Temperature 4.0 validation perplexity after epoch 2: 2533.04
Temperature 4.0: epoch 3/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 4.0 average training loss after epoch 3: 2.459
Temperature 4.0 validation perplexity after epoch 3: 1756.62
Temperature 4.0: epoch 4/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 4.0 average training loss after epoch 4: 2.263
Temperature 4.0 validation perplexity after epoch 4: 1551.50
Temperature 4.0: epoch 5/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 4.0 average training loss after epoch 5: 2.167
Temperature 4.0 validation perplexity after epoch 5: 1302.20
Restored Temperature 4.0 to epoch 5 (validation perplexity 1302.20)
Finished training Temperature 4.0 model.

=== Temperature sweep: T = 7.0 ===

Temperature 7.0: epoch 1/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 7.0 average training loss after epoch 1: 3.147
Temperature 7.0 validation perplexity after epoch 1: 3938.36
Temperature 7.0: epoch 2/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 7.0 average training loss after epoch 2: 2.449
Temperature 7.0 validation perplexity after epoch 2: 2634.50
Temperature 7.0: epoch 3/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 7.0 average training loss after epoch 3: 2.111
Temperature 7.0 validation perplexity after epoch 3: 2030.64
Temperature 7.0: epoch 4/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 7.0 average training loss after epoch 4: 1.991
Temperature 7.0 validation perplexity after epoch 4: 1720.85
Temperature 7.0: epoch 5/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 7.0 average training loss after epoch 5: 1.901
Temperature 7.0 validation perplexity after epoch 5: 1516.87
Restored Temperature 7.0 to epoch 5 (validation perplexity 1516.87)
Finished training Temperature 7.0 model.

=== Temperature sweep: T = 10.0 ===

Temperature 10.0: epoch 1/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 10.0 average training loss after epoch 1: 3.148
Temperature 10.0 validation perplexity after epoch 1: 4524.13
Temperature 10.0: epoch 2/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 10.0 average training loss after epoch 2: 2.539
Temperature 10.0 validation perplexity after epoch 2: 3194.35
Temperature 10.0: epoch 3/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 10.0 average training loss after epoch 3: 2.282
Temperature 10.0 validation perplexity after epoch 3: 3004.88
Temperature 10.0: epoch 4/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 10.0 average training loss after epoch 4: 2.080
Temperature 10.0 validation perplexity after epoch 4: 2497.72
Temperature 10.0: epoch 5/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 10.0 average training loss after epoch 5: 1.948
Temperature 10.0 validation perplexity after epoch 5: 2164.12
Restored Temperature 10.0 to epoch 5 (validation perplexity 2164.12)
Finished training Temperature 10.0 model.

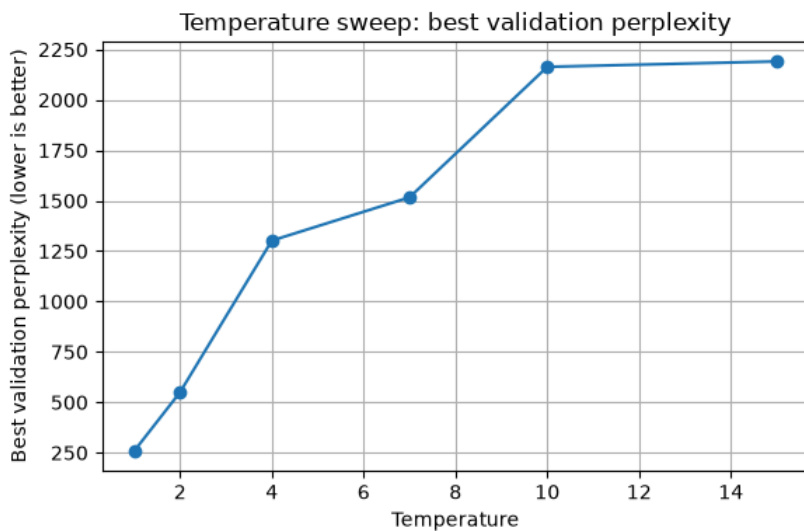
=== Temperature sweep: T = 15.0 ===

Temperature 15.0: epoch 1/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 15.0 average training loss after epoch 1: 2.957
Temperature 15.0 validation perplexity after epoch 1: 5483.90
Temperature 15.0: epoch 2/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 15.0 average training loss after epoch 2: 2.340
Temperature 15.0 validation perplexity after epoch 2: 3537.02
Temperature 15.0: epoch 3/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 15.0 average training loss after epoch 3: 2.133
Temperature 15.0 validation perplexity after epoch 3: 3056.56
Temperature 15.0: epoch 4/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 15.0 average training loss after epoch 4: 1.964
Temperature 15.0 validation perplexity after epoch 4: 2528.43
Temperature 15.0: epoch 5/5: 0%| | 0/145 [00:00<?, ?it/s]
Temperature 15.0 average training loss after epoch 5: 1.873
Temperature 15.0 validation perplexity after epoch 5: 2190.88
Restored Temperature 15.0 to epoch 5 (validation perplexity 2190.88)
Finished training Temperature 15.0 model.

Best temperature from sweep: 1.0

Out[9]:	Temperature	Final training loss (KL x T^2)	Best validation perplexity	Final validation perplexity
0	1.0	2.185229	258.123115	258.123115
1	2.0	2.749672	549.957637	549.957637
2	4.0	2.167040	1302.203994	1302.203994
3	7.0	1.900946	1516.869938	1516.869938
4	10.0	1.947665	2164.115828	2164.115828
5	15.0	1.873398	2190.884306	2190.884306

```
In [10]: plt.figure(figsize=(6, 4))
plt.plot(
    temperature_sweep["Temperature"],
    temperature_sweep["Best validation perplexity"],
    marker="o",
)
plt.title("Temperature sweep: best validation perplexity")
plt.xlabel("Temperature")
plt.ylabel("Best validation perplexity (lower is better)")
plt.grid(True)
plt.tight_layout()
os.makedirs("results", exist_ok=True)
plt.savefig("results/temperature_sweep.png", dpi=150)
plt.show()
```



7.4 Train All 3 Student Models

- Teacher + corpus
- Corpus only
- Teacher only

```
In [11]: # Train all three final student variants using the best temperature from the sweep.
print(f"Final student training will use temperature = {CFG['temperature']}")

training_methods = {
    "Teacher + corpus": "combined",
    "Corpus only": "corpus_only",
    "Teacher only": "teacher_only",
}

student_loss_history = {}
student_validation_history = {}

for name, method in training_methods.items():
    print(f"=== Training: {name} ===")
    loss_history, validation_history = train_model(student_models[name], name, method)
    student_loss_history[name] = loss_history
    student_validation_history[name] = validation_history
```

Final student training will use temperature = 1.0

```
=== Training: Teacher + corpus ===
Teacher + corpus: epoch 1/50: 0% | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 1: 5.804
Teacher + corpus validation perplexity after epoch 1: 1008.77
Teacher + corpus: epoch 2/50: 0% | 0/145 [00:00<?, ?it/s]
```

Teacher + corpus average training loss after epoch 2: 4.992
Teacher + corpus validation perplexity after epoch 2: 579.92
Teacher + corpus: epoch 3/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 3: 4.568
Teacher + corpus validation perplexity after epoch 3: 445.00
Teacher + corpus: epoch 4/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 4: 4.314
Teacher + corpus validation perplexity after epoch 4: 371.17
Teacher + corpus: epoch 5/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 5: 4.113
Teacher + corpus validation perplexity after epoch 5: 317.12
Teacher + corpus: epoch 6/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 6: 3.913
Teacher + corpus validation perplexity after epoch 6: 271.47
Teacher + corpus: epoch 7/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 7: 3.718
Teacher + corpus validation perplexity after epoch 7: 233.28
Teacher + corpus: epoch 8/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 8: 3.535
Teacher + corpus validation perplexity after epoch 8: 203.50
Teacher + corpus: epoch 9/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 9: 3.369
Teacher + corpus validation perplexity after epoch 9: 182.64
Teacher + corpus: epoch 10/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 10: 3.218
Teacher + corpus validation perplexity after epoch 10: 166.79
Teacher + corpus: epoch 11/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 11: 3.088
Teacher + corpus validation perplexity after epoch 11: 151.90
Teacher + corpus: epoch 12/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 12: 2.973
Teacher + corpus validation perplexity after epoch 12: 144.86
Teacher + corpus: epoch 13/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 13: 2.869
Teacher + corpus validation perplexity after epoch 13: 134.50
Teacher + corpus: epoch 14/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 14: 2.775
Teacher + corpus validation perplexity after epoch 14: 130.07
Teacher + corpus: epoch 15/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 15: 2.689
Teacher + corpus validation perplexity after epoch 15: 125.28
Teacher + corpus: epoch 16/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 16: 2.618
Teacher + corpus validation perplexity after epoch 16: 121.30
Teacher + corpus: epoch 17/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 17: 2.553
Teacher + corpus validation perplexity after epoch 17: 119.58
Teacher + corpus: epoch 18/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 18: 2.490
Teacher + corpus validation perplexity after epoch 18: 118.16
Teacher + corpus: epoch 19/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 19: 2.437
Teacher + corpus validation perplexity after epoch 19: 115.04
Teacher + corpus: epoch 20/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 20: 2.386
Teacher + corpus validation perplexity after epoch 20: 114.07
Teacher + corpus: epoch 21/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 21: 2.340
Teacher + corpus validation perplexity after epoch 21: 112.72
Teacher + corpus: epoch 22/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 22: 2.298
Teacher + corpus validation perplexity after epoch 22: 111.18
Teacher + corpus: epoch 23/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 23: 2.262
Teacher + corpus validation perplexity after epoch 23: 111.75
Teacher + corpus: epoch 24/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher + corpus average training loss after epoch 24: 2.225
Teacher + corpus validation perplexity after epoch 24: 112.33
Early stopping Teacher + corpus: validation perplexity did not improve.
Restored Teacher + corpus to epoch 22 (validation perplexity 111.18)
Finished training Teacher + corpus model.

=== Training: Corpus only ===

Corpus only: epoch 1/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 1: 7.473
Corpus only validation perplexity after epoch 1: 954.15
Corpus only: epoch 2/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 2: 6.563
Corpus only validation perplexity after epoch 2: 589.39
Corpus only: epoch 3/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 3: 6.142
Corpus only validation perplexity after epoch 3: 450.71
Corpus only: epoch 4/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 4: 5.845
Corpus only validation perplexity after epoch 4: 386.44

Corpus only: epoch 5/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 5: 5.616
Corpus only validation perplexity after epoch 5: 345.29
Corpus only: epoch 6/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 6: 5.405
Corpus only validation perplexity after epoch 6: 312.25
Corpus only: epoch 7/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 7: 5.196
Corpus only validation perplexity after epoch 7: 279.31
Corpus only: epoch 8/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 8: 4.982
Corpus only validation perplexity after epoch 8: 258.88
Corpus only: epoch 9/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 9: 4.777
Corpus only validation perplexity after epoch 9: 239.14
Corpus only: epoch 10/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 10: 4.591
Corpus only validation perplexity after epoch 10: 227.30
Corpus only: epoch 11/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 11: 4.417
Corpus only validation perplexity after epoch 11: 217.84
Corpus only: epoch 12/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 12: 4.254
Corpus only validation perplexity after epoch 12: 219.92
Corpus only: epoch 13/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 13: 4.102
Corpus only validation perplexity after epoch 13: 214.45
Corpus only: epoch 14/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 14: 3.964
Corpus only validation perplexity after epoch 14: 215.07
Corpus only: epoch 15/50: 0%| | 0/145 [00:00<?, ?it/s]
Corpus only average training loss after epoch 15: 3.831
Corpus only validation perplexity after epoch 15: 219.54
Early stopping Corpus only: validation perplexity did not improve.
Restored Corpus only to epoch 13 (validation perplexity 214.45)
Finished training Corpus only model.

=== Training: Teacher only ===

Teacher only: epoch 1/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 1: 3.891
Teacher only validation perplexity after epoch 1: 735.70
Teacher only: epoch 2/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 2: 2.986
Teacher only validation perplexity after epoch 2: 478.31
Teacher only: epoch 3/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 3: 2.636
Teacher only validation perplexity after epoch 3: 382.63
Teacher only: epoch 4/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 4: 2.400
Teacher only validation perplexity after epoch 4: 311.82
Teacher only: epoch 5/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 5: 2.185
Teacher only validation perplexity after epoch 5: 257.95
Teacher only: epoch 6/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 6: 1.980
Teacher only validation perplexity after epoch 6: 219.29
Teacher only: epoch 7/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 7: 1.813
Teacher only validation perplexity after epoch 7: 191.55
Teacher only: epoch 8/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 8: 1.666
Teacher only validation perplexity after epoch 8: 171.04
Teacher only: epoch 9/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 9: 1.546
Teacher only validation perplexity after epoch 9: 155.21
Teacher only: epoch 10/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 10: 1.443
Teacher only validation perplexity after epoch 10: 146.62
Teacher only: epoch 11/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 11: 1.352
Teacher only validation perplexity after epoch 11: 136.71
Teacher only: epoch 12/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 12: 1.275
Teacher only validation perplexity after epoch 12: 130.25
Teacher only: epoch 13/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 13: 1.211
Teacher only validation perplexity after epoch 13: 123.23
Teacher only: epoch 14/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 14: 1.153
Teacher only validation perplexity after epoch 14: 120.02
Teacher only: epoch 15/50: 0%| | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 15: 1.102
Teacher only validation perplexity after epoch 15: 116.12
Teacher only: epoch 16/50: 0%| | 0/145 [00:00<?, ?it/s]

```

Teacher only average training loss after epoch 16: 1.059
Teacher only validation perplexity after epoch 16: 114.19
Teacher only: epoch 17/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 17: 1.019
Teacher only validation perplexity after epoch 17: 111.99
Teacher only: epoch 18/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 18: 0.985
Teacher only validation perplexity after epoch 18: 109.11
Teacher only: epoch 19/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 19: 0.953
Teacher only validation perplexity after epoch 19: 106.85
Teacher only: epoch 20/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 20: 0.924
Teacher only validation perplexity after epoch 20: 105.35
Teacher only: epoch 21/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 21: 0.899
Teacher only validation perplexity after epoch 21: 104.30
Teacher only: epoch 22/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 22: 0.875
Teacher only validation perplexity after epoch 22: 101.85
Teacher only: epoch 23/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 23: 0.854
Teacher only validation perplexity after epoch 23: 100.84
Teacher only: epoch 24/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 24: 0.834
Teacher only validation perplexity after epoch 24: 100.14
Teacher only: epoch 25/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 25: 0.816
Teacher only validation perplexity after epoch 25: 98.93
Teacher only: epoch 26/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 26: 0.801
Teacher only validation perplexity after epoch 26: 97.66
Teacher only: epoch 27/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 27: 0.783
Teacher only validation perplexity after epoch 27: 96.28
Teacher only: epoch 28/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 28: 0.768
Teacher only validation perplexity after epoch 28: 95.91
Teacher only: epoch 29/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 29: 0.757
Teacher only validation perplexity after epoch 29: 95.23
Teacher only: epoch 30/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 30: 0.741
Teacher only validation perplexity after epoch 30: 94.58
Teacher only: epoch 31/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 31: 0.729
Teacher only validation perplexity after epoch 31: 94.46
Teacher only: epoch 32/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 32: 0.719
Teacher only validation perplexity after epoch 32: 93.70
Teacher only: epoch 33/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 33: 0.709
Teacher only validation perplexity after epoch 33: 92.58
Teacher only: epoch 34/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 34: 0.699
Teacher only validation perplexity after epoch 34: 93.76
Teacher only: epoch 35/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 35: 0.692
Teacher only validation perplexity after epoch 35: 92.06
Teacher only: epoch 36/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 36: 0.682
Teacher only validation perplexity after epoch 36: 91.61
Teacher only: epoch 37/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 37: 0.673
Teacher only validation perplexity after epoch 37: 91.00
Teacher only: epoch 38/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 38: 0.665
Teacher only validation perplexity after epoch 38: 90.22
Teacher only: epoch 39/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 39: 0.658
Teacher only validation perplexity after epoch 39: 89.44
Teacher only: epoch 40/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 40: 0.649
Teacher only validation perplexity after epoch 40: 90.00
Teacher only: epoch 41/50: 0%|          | 0/145 [00:00<?, ?it/s]
Teacher only average training loss after epoch 41: 0.644
Teacher only validation perplexity after epoch 41: 90.01
Early stopping Teacher only: validation perplexity did not improve.
Restored Teacher only to epoch 39 (validation perplexity 89.44)
Finished training Teacher only model.

```

8. Save The Trained Student Models

We save each trained student separately so the three methods can be reloaded or inspected later.

```
In [12]: checkpoint_dirs = {
    "Teacher + corpus": "checkpoints/student_teacher_plus_corpus",
    "Corpus only": "checkpoints/student_corpus_only",
    "Teacher only": "checkpoints/student_teacher_only",
}

for name, path in checkpoint_dirs.items():
    os.makedirs(path, exist_ok=True)
    student_models[name].save_pretrained(path)
    tokenizer.save_pretrained(path)
    print(f"Saved {name} -> {path}")
```

```
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
Saved Teacher + corpus -> checkpoints/student_teacher_plus_corpus
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
Saved Corpus only -> checkpoints/student_corpus_only
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
Saved Teacher only -> checkpoints/student_teacher_only
```

9. Evaluation & Analysis

We now compare the **teacher** and all **three student training methods**.

9.1 Validation and test perplexity

Lower is better. Validation perplexity helps check overfitting, while test perplexity is the final performance number.

```
In [13]: # Measure validation and test perplexity.
teacher_val_ppl = evaluate_perplexity(teacher, val_loader)
teacher_ppl = evaluate_perplexity(teacher, test_loader)

student_val_ppls = {}
student_ppls = {}

for name, model in student_models.items():
    student_val_ppls[name] = evaluate_perplexity(model, val_loader)
    student_ppls[name] = evaluate_perplexity(model, test_loader)

print(f"Teacher validation perplexity: {teacher_val_ppl:.2f}")
print(f"Teacher test perplexity: {teacher_ppl:.2f}")

for name in student_models:
    print(f"{name} validation perplexity: {student_val_ppls[name]:.2f}")
    print(f"{name} test perplexity: {student_ppls[name]:.2f}")
```

```
Teacher validation perplexity: 30.63
Teacher test perplexity: 29.37
Teacher + corpus validation perplexity: 111.18
Teacher + corpus test perplexity: 115.64
Corpus only validation perplexity: 214.45
Corpus only test perplexity: 227.19
Teacher only validation perplexity: 89.44
Teacher only test perplexity: 90.72
```

9.2 Generated text quality

We feed the **same prompts** with deterministic decoding to the teacher and each student, then compare fluency, coherence, and repetition.

```
In [14]: def generate_text(model, prompt, max_new_tokens=40):
    model.eval()
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)
    with torch.no_grad():
        output_ids = model.generate(
            input_ids,
            max_new_tokens=max_new_tokens,
            do_sample=False, # deterministic generation for fair comparison
            pad_token_id=tokenizer.eos_token_id,
        )
    return tokenizer.decode(output_ids[0], skip_special_tokens=True)

all_models = {"Teacher (GPT-2)": teacher, **student_models}

prompts = [
    "The history of artificial intelligence",
    "The city was famous for its",
    "Climate change is one of the most important",
    "A good machine learning model should",
    "The scientist opened the notebook and discovered",
    "In simple terms, gravity is",
    "The main advantage of renewable energy is",
    "During the experiment, the researchers observed",
```

```
    "Once upon a time in a small village",  
]  
  
for prompt in prompts:  
    print("=" * 90)  
    print(f"PROMPT: {prompt}\n")  
    for name, model in all_models.items():  
        print(f"[{name}] {generate_text(model, prompt)}\n")
```

[transformers] The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

PROMPT: The history of artificial intelligence

[Teacher (GPT-2)] The history of artificial intelligence is littered with examples of how it has been used to create a world where people are more likely to be able to do things that are not possible in the real world.

The problem is that

[Teacher + corpus] The history of artificial intelligence, the world was a very popularized by the world . The first game was the first game to be played by the game in the series . The game was the first game in the series of the

[Corpus only] The history of artificial intelligence , and the main history of the Dominican Republic of the Dominican Republic of the Dominican R
epublic of the Dominican Republic . The Dominican Republic of the Dominican Republic of the Dominican Republic of the Dominican Republic of the Do
minican

[Teacher only] The history of artificial intelligence, and the world is not a human being, but it is a human being, and it is a human being a human being, and a human being, and a human being, and a human

PROMPT: The city was famous for its

[Teacher (GPT-2)] The city was famous for its high-speed rail system, which was built in the 1930s.

The city's first subway was built in the 1930s, but it was shut down in the 1940s.

[illegible]

[Corpus only] The city was famous for its city . The city 's city 's city is a city , and the city 's city , and the city 's city , and the city , and the city , and the city ' .

[Teacher only] The city was famous for its "greatest and most beautiful and beautiful, but also a great example of the most beautiful and beautiful, and beautiful, and beautiful.

PROMPT: Climate change is one of the most important

[Teacher (GPT-2)] Climate change is one of the most important issues facing the world today.

The world is facing a major crisis of climate change. The world is facing a major crisis of climate change.

The world is facing a major crisis of

[Teacher + corpus] Climate change is one of the most important part of the past three years of the past decade . The report was published in the report of the report of the report of the report , which was published in the report of the report . The report

[Corpus only] Climate change is one of the most important role of the most of the island .

== == == == == == ==

The island of the island is a large island of the island of the island of the island . The island

[Teacher only] Climate change is one of the most important things in the world.

=====

PROMPT: A good machine learning model should

[Teacher (GPT-2)] A good machine learning model should be able to predict the future.

The problem is that the model is not very good at predicting the future. It is not very good at predicting the future. It is not very good at

[Teacher + corpus] A good machine learning model should be used to be used to be used in the Australian Australian Australian Australian Australia
n Australian Australian Australian Australian Australian Australian Australian Australian Australian Australian Australian Australian A
ustralian Australian Australian Australian Australian Australian Australian Australian Australian Australian Australian Australian

[Corpus only] A good machine learning model should be used to be used to be used to be used to be used to be used to be used to be used to be used
in the project . The project was the project to be used in the

[Teacher only] A good machine learning model should be used to make a new model for the game.com.

=====

PROMPT: The scientist opened the notebook and discovered

[Teacher (GPT-2)] The scientist opened the notebook and discovered that the paper was filled with a number of different types of bacteria.

"I was surprised to find that the bacteria were all different," he said. "I was surprised to find that the

[Teacher + corpus] The scientist opened the notebook and discovered the discovery of the discovery of the discovery of the moon of the moon of the
moon . The first moon was discovered in the moon of the moon , and the moon was discovered in the moon . The

[Corpus only] The scientist opened the notebook and discovered in the 18th century . The first century , and the first century , and the first cen
tury , and the first century , and the first century , and the first century , and the first century ,

[Teacher only] The scientist opened the notebook and discovered the first book of the book, and then published in the journal of the journal.S. Th
e New York Times. (19-19-19) . The book was published in the journal of

=====

PROMPT: In simple terms, gravity is

[Teacher (GPT-2)] In simple terms, gravity is the force of gravity.

The force of gravity is the force of gravity on a solid object.

The force of gravity is the force of gravity on a solid object.

The

[Teacher + corpus] In simple terms, gravity is not the same . The song is a "a " , which is a "a " , " a song that is "a " , " a song that is " a song that is " a

[Corpus only] In simple terms, gravity is a number of the Sun 's main sequence of the Sun 's atmosphere , which is a massive star , with a star 's atmosphere , which is a star , which is a star , which

[Teacher only] In simple terms, gravity is a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and a simple, and

=====
PROMPT: The main advantage of renewable energy is

[Teacher (GPT-2)] The main advantage of renewable energy is that it can be used to power the grid, and that's what we're doing here.

"We're not going to be able to do that with wind and solar. We're going

[Teacher + corpus] The main advantage of renewable energy is the most common cause of the world . The most common cause of the world is the most common cause of the world . The most common cause of the loss of the world is the most common cause of

[Corpus only] The main advantage of renewable energy is the gliding and the gliding and the gliding is the gliding and a gliding , and is the gliding , and a gliding , and is the gliding , and gliding

[Teacher only] The main advantage of renewable energy is the most expensive energy in the world.

=====
PROMPT: During the experiment, the researchers observed

[Teacher (GPT-2)] During the experiment, the researchers observed that the brain's ability to process information was impaired when the brain was exposed to a high-energy laser beam.

"The laser beam was very bright and very bright, and it was very

[Teacher + corpus] During the experiment, the researchers observed the experiment of the experiment . The experiment was later published in the Los Angeles Times in Los Angeles , California , and the Los Angeles Times published a report on the report on the report . The report was

[Corpus only] During the experiment, the researchers observed that the Sun 's body was observed in the Sun 's body .
= = = = =
The first known as a number of the Sun 's body was a number

[Teacher only] During the experiment, the researchers observed that the problem of the problem was that the problem was not the problem for the problem.comatical problems.

=====

PROMPT: Once upon a time in a small village

[Teacher (GPT-2)] Once upon a time in a small village, a young girl named Nana was kidnapped by a group of bandits. She was taken to a small village, where she was forced to work as a maid. She was then taken to a small

[Teacher + corpus] Once upon a time in a small village of the village of the village of the village of the village of the village of the village. The village of the village of the village of St. John 's village. John 's village.

[Corpus only] Once upon a time in a small village .

= = = = =

The first phase of the area of the area of the area of the area of the area of the area of the area of the area of the area of the

[Teacher only] Once upon a time in a small village, and the village's the village's had been evacuated to the village.

9.3 Inference speed

We time how long each model takes to generate a fixed number of tokens and report **tokens per second**. Higher is better.

```
In [15]: def measure_speed(model, n_tokens=100, n_runs=5):
model.eval()
input_ids = tokenizer("The", return_tensors="pt").input_ids.to(device)

def one_run():
    with torch.no_grad():
        model.generate(
            input_ids,
            max_new_tokens=n_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
        )

# The first generation call is often slower than normal because PyTorch/GPU may need to do one-time setup
one_run()
if device == "cuda":
    torch.cuda.synchronize() # forces Python to wait until the GPU is done

start = time.time()
for _ in range(n_runs):
    one_run()
if device == "cuda":
    torch.cuda.synchronize()

elapsed = time.time() - start
return (n_tokens * n_runs) / elapsed

teacher_tps = measure_speed(teacher)
student_tps = {
```

```

    name: measure_speed(model)
    for name, model in student_models.items()
}

print(f"Teacher: {teacher_tps:.1f} tokens/sec")
for name, tps in student_tps.items():
    print(f"{name}: {tps:.1f} tokens/sec ({tps / teacher_tps:.1f}x teacher speed)")

```

Teacher: 164.6 tokens/sec
Teacher + corpus: 295.1 tokens/sec (1.8x teacher speed)
Corpus only: 294.3 tokens/sec (1.8x teacher speed)
Teacher only: 292.3 tokens/sec (1.8x teacher speed)

9.4 Model size & parameter count

Finally we report parameter counts, on-disk size, perplexity, and speed in one summary table.

```

In [16]: def dir_size_mb(path) -> float:
    """Total size (in MB) of the saved-model files in a folder."""
    total_bytes = 0
    for root, _dirs, files in os.walk(path):
        for f in files:
            total_bytes += os.path.getsize(os.path.join(root, f))
    return total_bytes / (1024 ** 2)

teacher_size_mb = teacher_params * 4 / (1024 ** 2) # teacher model is not saved, so estimate: float32 weights

rows = [
    {
        "Model": "Teacher (GPT-2)",
        "Training method": "pretrained teacher",
        "Parameters (M)": teacher_params / 1e6,
        "Disk size (MB)": teacher_size_mb,
        "Validation perplexity": teacher_val_ppl,
        "Test perplexity": teacher_ppl,
        "Tokens/sec": teacher_tps,
    }
]

for name in student_models:
    rows.append(
        {
            "Model": f"Student ({name})",
            "Training method": training_methods[name],
            "Parameters (M)": student_params / 1e6,
            "Disk size (MB)": dir_size_mb(checkpoint_dirs[name]),
            "Validation perplexity": student_val_ppls[name],
            "Test perplexity": student_ppls[name],
            "Tokens/sec": student_tps[name],
        }
    )

summary = pd.DataFrame(rows).round(2)
display(summary)

```

	Model	Training method	Parameters (M)	Disk size (MB)	Validation perplexity	Test perplexity	Tokens/sec
0	Teacher (GPT-2)	pretrained teacher	124.44	474.70	30.63	29.37	164.57
1	Student (Teacher + corpus)	combined	30.34	119.14	111.18	115.64	295.09
2	Student (Corpus only)	corpus_only	30.34	119.14	214.45	227.19	294.31
3	Student (Teacher only)	teacher_only	30.34	119.14	89.44	90.72	292.31

10. Plots & Conclusions

Visualising the numbers makes the training-method comparison clearer. We plot:

1. Training loss per epoch for all three students.
2. Validation perplexity per epoch for all three students.
3. Final test perplexity for teacher and students.
4. Inference speed for teacher and students.

```

In [21]: os.makedirs("results", exist_ok=True)

fig, axes = plt.subplots(2, 2, figsize=(20, 12))
axes = axes.flatten()

max_epochs_plotted = max(len(losses) for losses in student_loss_history.values())

```

```

# (1) Training Loss per epoch. The three curves use different loss definitions, so they show each
# student's own progress but must not be compared with each other.
for name, losses in student_loss_history.items():
    epochs_axis = range(1, len(losses) + 1)
    axes[0].plot(epochs_axis, losses, marker="o", label=name)
axes[0].set_title("Training loss per epoch (scales differ per method)")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Average training loss")
axes[0].set_xticks(range(1, max_epochs_plotted + 1))
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# (2) Validation perplexity per epoch, on a raw (linear) scale. Measured identically for every
# method, so this is the curve to compare across students.
for name, ppls in student_validation_history.items():
    epochs_axis = range(1, len(ppls) + 1)
    line, = axes[1].plot(epochs_axis, ppls, marker="o", label=name)
    best_epoch = int(np.argmin(ppls)) + 1
    axes[1].scatter(
        [best_epoch], [min(ppls)],
        s=160, facecolors="none", edgecolors=line.get_color(), linewidths=2, zorder=5,
    )
axes[1].axhline(teacher_val_ppl, color="gray", linestyle="--", linewidth=1.5, label="Teacher (GPT-2)")
axes[1].set_title("Validation perplexity per epoch (lower is better)")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Validation perplexity")
axes[1].set_xticks(range(1, max_epochs_plotted + 1))
axes[1].grid(True, alpha=0.3)
axes[1].legend()

# (3) Final test perplexity.
plot_names = ["Teacher"] + list(student_models.keys())
plot_ppl = [teacher_ppl] + [student_ppls[name] for name in student_models]
axes[2].bar(plot_names, plot_ppl)
axes[2].set_title("Test perplexity (lower is better)")
axes[2].set_ylabel("Perplexity")
axes[2].tick_params(axis="x", rotation=25)

# (4) Inference speed.
plot_tps = [teacher_tps] + [student_tps[name] for name in student_models]
axes[3].bar(plot_names, plot_tps)
axes[3].set_title("Inference speed (higher is better)")
axes[3].set_ylabel("Tokens / sec")
axes[3].tick_params(axis="x", rotation=25)

plt.tight_layout()
plt.savefig("results/comparison.png", dpi=150)
plt.show()

```

